

CHAPTER

8

Transformers

“The true art of memory is the art of attention ”

Samuel Johnson, *Idler* #74, September 1759

In this chapter we introduce the **transformer**, the standard architecture for building large language models. As we discussed in the prior chapter, transformer-based large language models have completely changed the field of speech and language processing. Indeed, every subsequent chapter in this textbook will make use of them. As with the previous chapter, we’ll focus for this chapter on the use of transformers to model left-to-right (sometimes called **causal** or autoregressive) language modeling, in which we are given a sequence of input tokens and predict output tokens one by one by conditioning on the prior context.

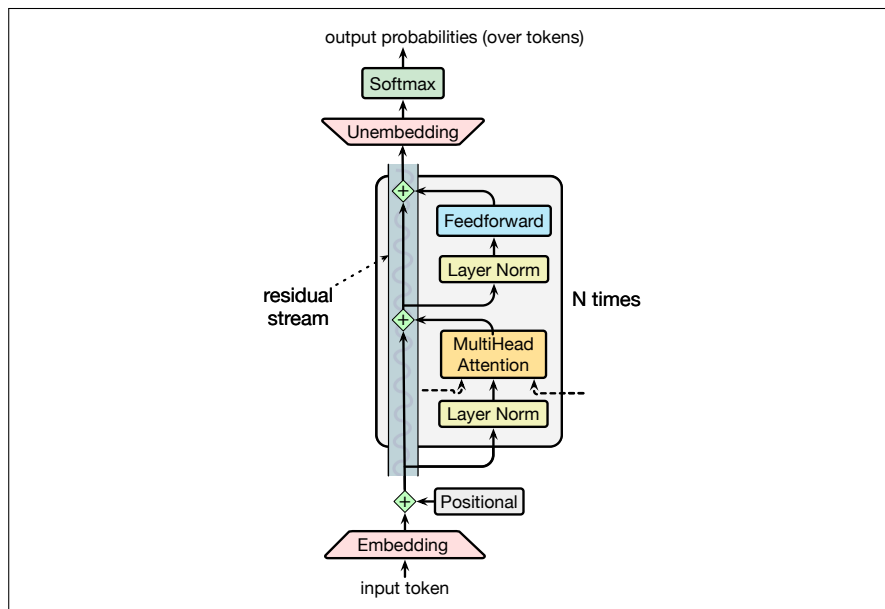


Figure 8.1 A transformer decoder for language modeling, showing the *residual stream* for processing an input token. A single token is embedded and passed forward in the network, with the feedforward and attention components adding information. The multihead attention layer takes inputs (not shown in detail) from the neighboring token streams. This is thus one column of an autoregressive transformer language model, taking an input token and outputting a distribution over next tokens.

Fig. 8.1 sketches the transformer architecture following a single token as it passes up through the layers of the network. Each token is first converted to an embedding from the **embedding matrix E**. Recall from Chapter 6 in Section ?? that **E** is a linear layer that maps a token id to a vector **embedding** representing that token. Each token in the vocabulary has an initial embedding representation in **E**.

Transformers also have a special mechanism for encoding the position/index of the token in the input string, which is simply added to the embedding. The resulting embedding represents both the word and its position. and is then passed through a set of N transformer blocks.

It's common to think of each of these transformer blocks as part of a **stream** in which the input embedding is directly passed up to the output, while simultaneously being enriched by the application of various processing modules: the **multi-head attention** layer, feedforward networks and the layer normalization. The value of the stream at any layer is the sum of the original embedding and all the outputs from all the previous layers and blocks.

The core intuition of the transformer, and the component that distinguishes it from the feedforward layers we saw in Chapter 6, is this multi-head attention layer, also called a **self-attention** layer. Attention can be thought of as a way to build contextual representations of a token's meaning by **attending to** and integrating information from surrounding tokens, helping the model learn how tokens relate to each other over large spans. It can also be thought of as a way to move information from one residual stream to another, augmenting the stream at one token position with information from another token position.

After the N transformer blocks we take the output embedding that is produced by the final transformer block, pass it through an linear **unembedding matrix** $\mathbf{U}$ and then a softmax over the vocabulary to generate a distribution over possible next tokens. These last two components (the unembedding matrix and the softmax) are sometimes called the **language modeling head**. In the rest of this chapter we'll introduce attention and the rest of these modules in more detail.

Fig. ?? shows the transformer architecture applied to a context window with the words `So long and thanks for`, showing at each token position what is the most likely token to be generated. In this full figure, the set of N blocks maps an entire **context window** of input vectors $(\mathbf{x}_1, \dots, \mathbf{x}_n)$ to a window of output vectors $(\mathbf{h}_1, \dots, \mathbf{h}_n)$ of the same length. A column might contain from 12 to 96 or more stacked blocks. The arrows in the figure shows how information from the hidden representations of preceding tokens is incorporated into the transformer block.

Transformer-based language models are complex, and so the details will unfold over this chapter and the next few chapters. Chapter 7 already discussed how language models are **pretrained**, and how tokens are generated via **sampling**. In the rest of this chapter we'll introduce multi-head attention, the rest of the transformer block, and the input encoding and language modeling head components of the transformer. Chapter 9 introduces **masked language modeling** and the **BERT** family of bidirectional transformer encoder models. Chapter 10 shows how to **instruction-tune** language models to perform NLP tasks, and how to **align** the model with human preferences. Chapter 12 will introduce machine translation with the **encoder-decoder** architecture. And we'll see application of the transformer to speech recognition, as well as further use of the encoder-decoder architecture, in Chapter 15.

8.1 Attention

Recall from Chapter 5 that for **word2vec** and other static embeddings, the representation of a word's meaning is always the same vector irrespective of the context: the word `chicken`, for example, is always represented by the same fixed vector. So a static vector for the word `it` might somehow encode that this is a pronoun used

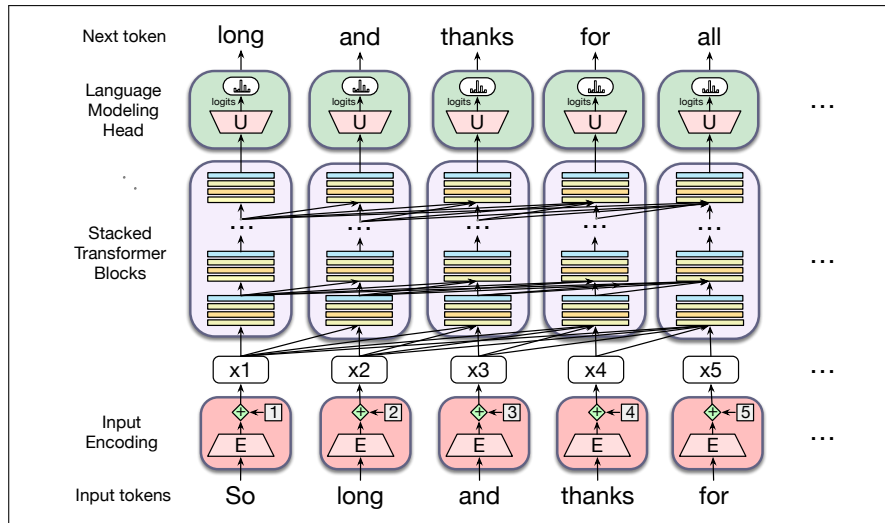


Figure 8.2 The architecture of a (left-to-right) transformer, showing how each input token get encoded, passed through a set of stacked transformer blocks, and then a language model head that predicts the next token. The embeddings at each token position in the residual stream are passed up the stack, and the arrows in the figure shows how information from the hidden representations of preceding tokens are also incorporated.

for animals and inanimate entities. But in context it has a much richer meaning. Consider it in one of these two sentences:

- (8.1) The **chicken** didn't cross the road because **it** was too tired.
 (8.2) The chicken didn't cross the **road** because **it** was too wide.

In (8.1) it is the chicken (i.e., the reader knows that the chicken was tired), while in (8.2) it is the road (and the reader knows that the road was wide).¹ That is, if we are to compute the meaning of this sentence, we'll need the meaning of it to be associated with the **chicken** in the first sentence and associated with the **road** in the second one, sensitive to the context.

Furthermore, consider reading left to right like a causal language model, processing the sentence up to the word it:

- (8.3) The **chicken** didn't cross the **road** because **it**

At this point we don't yet know which thing it is going to end up referring to! So a representation of it at this point might have aspects of both **chicken** and **road** as the reader is trying to guess what happens next.

This fact that words have rich linguistic relationships with other words that may be far away pervades language. Consider two more examples:

- (8.4) The **keys** to the cabinet **are** on the table.
 (8.5) I walked along the **pond**, and noticed one of the trees along the **bank**.

In (8.4), the phrase *The keys* is the subject of the sentence, and in English and many languages, must agree in grammatical number with the verb *are*; in this case both are plural. In English we can't use a singular verb like *is* with a plural subject like *keys* (we'll discuss agreement more in Chapter 18). In (8.5), we know that *bank* refers to the side of a pond or river and not a financial institution because of the context, including words like *pond*. (We'll discuss word senses more in Chapter 9.)

¹ We say that in the first example it **corefers** with the chicken, and in the second it corefers with the road; we'll return to this in Chapter 23.

contextual
embeddings

The point of all these examples is that these contextual words that help us compute the meaning of words in context can be quite far away in the sentence or paragraph. Transformers can build contextual representations of word meaning, **contextual embeddings**, by integrating the meaning of these helpful contextual words. In a transformer, layer by layer, we build up richer and richer contextualized representations of the meanings of input tokens. At each layer, we compute the representation of a token i by combining information about i from the previous layer with information about the neighboring tokens to produce a contextualized representation for each word at each position.

Attention is the mechanism in the transformer that weighs and combines the representations from appropriate other tokens in the context from layer k to build the representation for tokens in layer $k + 1$.

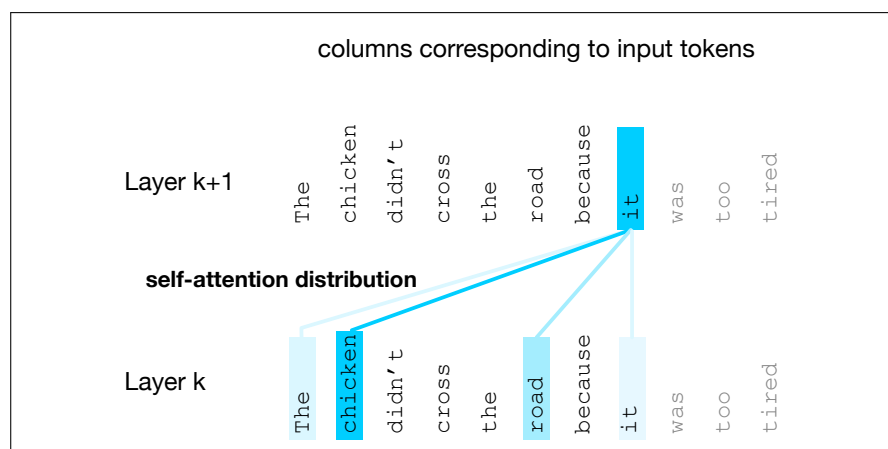


Figure 8.3 The self-attention weight distribution α that is part of the computation of the representation for the word *it* at layer $k + 1$. In computing the representation for *it*, we attend differently to the various words at layer k , with darker shades indicating higher self-attention values. Note that the transformer is attending highly to the columns corresponding to the tokens *chicken* and *road*, a sensible result, since at the point where *it* occurs, it could plausibly corefer with the chicken or the road, and hence we'd like the representation for *it* to draw on the representation for these earlier words. Figure adapted from [Uszkoreit \(2017\)](#).

Fig. 8.3 shows a schematic example simplified from a transformer ([Uszkoreit, 2017](#)). The figure describes the situation when the current token is *it* and we need to compute a contextual representation for this token at layer $k + 1$ of the transformer, drawing on the representations (from layer k) of every prior token. The figure uses color to represent the attention distribution over the contextual words: the tokens *chicken* and *road* both have a high attention weight, meaning that as we are computing the representation for *it*, we will draw most heavily on the representation for *chicken* and *road*. This will be useful in building the final representation for *it*, since *it* will end up coreferring with either *chicken* or *road*.

Let's now turn to how this attention distribution is represented and computed.

8.1.1 Attention more formally

As we've said, the attention computation is a way to compute a vector representation for a token at a particular layer of a transformer, by selectively attending to and integrating information from prior tokens at the previous layer. Attention takes an

input representation $\mathbf{x}_i$ corresponding to the input token at position i , and a context window of prior inputs $\mathbf{x}_1 \dots \mathbf{x}_{i-1}$, and produces an output $\mathbf{a}_i$.

In causal, left-to-right language models, the context is any of the prior words. That is, when processing $\mathbf{x}_i$, the model has access to $\mathbf{x}_i$ as well as the representations of all the prior tokens in the context window (context windows consist of thousands of tokens) but no tokens after i . (By contrast, in Chapter 9 we'll generalize attention so it can also look ahead to future words.)

Fig. 8.4 illustrates this flow of information in an entire causal self-attention layer, in which this same attention computation happens in parallel at each token position i . Thus a self-attention layer maps input sequences $(\mathbf{x}_1, \dots, \mathbf{x}_n)$ to output sequences of the same length $(\mathbf{a}_1, \dots, \mathbf{a}_n)$.

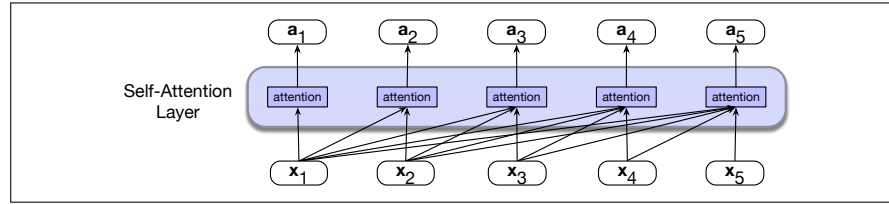


Figure 8.4 Information flow in causal self-attention. When processing each input $\mathbf{x}_i$, the model attends to all the inputs up to, and including $\mathbf{x}_i$.

Simplified version of attention At its heart, attention is really just a weighted sum of context vectors, with a lot of complications added to how the weights are computed and what gets summed. For pedagogical purposes let's first describe a simplified intuition of attention, in which the attention output $\mathbf{a}_i$ at token position i is simply the weighted sum of all the representations $\mathbf{x}_j$, for all $j \leq i$; we'll use α_{ij} to mean how much $\mathbf{x}_j$ should contribute to $\mathbf{a}_i$:

$$\text{Simplified version: } \mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j \quad (8.6)$$

Each α_{ij} is a scalar used for weighing the value of input $\mathbf{x}_j$ when summing up the inputs to compute $\mathbf{a}_i$. How shall we compute this α weighting? In attention we weight each prior embedding proportionally to how **similar** it is to the current token i . So the output of attention is a sum of the embeddings of prior tokens weighted by their similarity with the current token embedding. We compute similarity scores via **dot product**, which maps two vectors into a scalar value ranging from $-\infty$ to ∞ . The larger the score, the more similar the vectors that are being compared. We'll normalize these scores with a softmax to create the vector of weights $\alpha_{ij}, j \leq i$.

$$\text{Simplified Version: } \text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j \quad (8.7)$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i \quad (8.8)$$

Thus in Fig. 8.4 we compute $\mathbf{a}_3$ by computing three scores: $\mathbf{x}_3 \cdot \mathbf{x}_1$, $\mathbf{x}_3 \cdot \mathbf{x}_2$ and $\mathbf{x}_3 \cdot \mathbf{x}_3$, normalizing them by a softmax, and using the resulting probabilities as weights indicating each of their proportional relevance to the current position i . Of course, the softmax weight will likely be highest for $\mathbf{x}_i$, since $\mathbf{x}_i$ is very similar to itself, resulting in a high dot product. But other context words may also be similar to i , and the softmax will also assign some weight to those words. Then we use these weights as the α values in Eq. 8.6 to compute the weighted sum that is our $\mathbf{a}_3$.

The simplified attention in equations 8.6 – 8.8 demonstrates the attention-based approach to computing $\mathbf{a}_i$: compare the $\mathbf{x}_i$ to prior vectors, normalize those scores